

Fluxo de Cadastro e Processamento de Uploads

Este documento descreve o fluxo atual de cadastro de **Project** e de upload/processamento de arquivos no formato unificado.

Endpoints principais

- **POST /v1/projects** - cria um **Project**.
- **POST /v1/uploads** - endpoint unico de upload (multipart) com arquivo + metadados.
- **GET /v1/uploads/{uploadId}** - consulta status do upload.

Contrato do endpoint de upload

POST /v1/uploads

Content-Type: **multipart/form-data**

Partes obrigatorias:

- **file** (binario)
- **metadata** (JSON)

Exemplo de **metadata**:

```
{
  "filename": "resultado.pdf",
  "contentType": "application/pdf",
  "projectId": "11111111-1111-1111-1111-111111111111",
  "uploaderId": "user-123"
}
```

PROF

Resposta de sucesso (**201 Created**):

```
{
  "uploadId": "a9a92228-5463-4722-aa39-cb495a52c030",
  "presignedUrl": null,
  "s3Key": "uploads/dev/11111111-1111-1111-1111-111111111111/2026-04-03/user-123/a9a92228-5463-4722-aa39-cb495a52c030.pdf",
  "expiresInSeconds": 0
}
```

Regras de validacao

- Tipos aceitos: **application/pdf**, **image/png**, **image/jpg**, **image/jpeg**
- O **contentType** do **metadata** deve ser igual ao **contentType** do arquivo enviado

- Tamanho maximo do arquivo: 1GB

Configuracao relevante:

- `spring.servlet.multipart.max-file-size=1GB`
- `spring.servlet.multipart.max-request-size=1GB`

Respostas de erro padronizadas

Todos os erros de validacao/not found seguem o formato:

```
{
  "message": "...",
  "code": "...",
  "detail": "..."
}
```

Exemplos:

Content-Type invalido (400):

```
{
  "message": "Invalid upload request",
  "code": "INVALID_CONTENT_TYPE",
  "detail": "Only PDF, PNG, JPG or JPEG files are allowed"
}
```

Arquivo acima de 1GB (400):

```
{
  "message": "Invalid upload request",
  "code": "FILE_SIZE_EXCEEDED",
  "detail": "File size exceeds maximum allowed size of 1GB"
}
```

Upload nao encontrado (404):

```
{
  "message": "Resource not found",
  "code": "UPLOAD_NOT_FOUND",
  "detail": "Upload not found for id <uuid>"
}
```

Resumo do fluxo

1. Cliente cria um projeto com **POST /v1/projects**.
2. Cliente envia arquivo e metadados no **POST /v1/uploads** (multipart).
3. **UploadController** valida arquivo, tipo e tamanho.
4. **SingleUploadUseCase** gera **uploadId** e **s3Key**, envia o arquivo ao S3, salva **Upload** com status **COMPLETED** e publica evento no SQS.
5. **ScanWorker** consome evento, verifica idempotencia e valida disponibilidade do arquivo no S3.
6. Em sucesso, status do upload passa para **SCANNED_OK**.
7. Em falhas repetidas, a mensagem e enviada para a DLQ conforme **maxRetries**.

Diagrama de Sequencia (PlantUML)

```
@startuml
actor Client
participant "ProjectController" as PC
participant "UploadController" as UC
participant "SingleUploadUseCase" as SU
participant "S3" as S3
participant "DB (Upload/Project)" as DB
participant "SQS (queue)" as SQS
participant "ScanWorker" as SW

Client -> PC: POST /v1/projects {name, ownerId}
PC -> DB: save Project
DB --> PC: Project{id}
PC --> Client: 200 OK

Client -> UC: POST /v1/uploads (multipart)
note right
parts:
- file (binary)
- metadata JSON
end note

UC -> UC: valida file/contentType/tamanho
UC -> SU: execute(metadata + file)
SU -> S3: PutObject
S3 --> SU: ETag
SU -> DB: save Upload(status=COMPLETED)
DB --> SU: Upload saved
SU -> SQS: send UploadEvent(eventId=uploadId,...)
SQS --> SU: SendMessageResult
SU --> UC: UploadResponse
UC --> Client: 201 Created

== Assincrono: Worker consome SQS ==
SW -> SQS: ReceiveMessage
SQS --> SW: Message{body}
SW -> SW: idempotency check (ProcessedMessage)
SW -> S3: GetObject(s3Key)
S3 --> SW: object stream
```

```
SW -> DB: update Upload status (SCANNED_OK)
DB --> SW: OK
SW -> SQS: DeleteMessage
SQS --> SW: OK

alt processing failure and receiveCount >= maxRetries
    SW -> SQS: SendMessage(DLQ)
    SQS --> SW: OK
    SW -> SQS: DeleteMessage(original)
end

@enduml
```

Referencias

- Swagger UI: [/swagger-ui.html](#)
- OpenAPI JSON: [/v3/api-docs](#)
- Collection Postman local: [docs/postman/upload-service.postman_collection.json](#)